

Weighted BDI Data Flow: Confidence, Priority, and Diagnostics

Author: CL.Investigate1 (Computational Linguist) **Date:** 2026-05-25 **Status:** Reference document

Overview

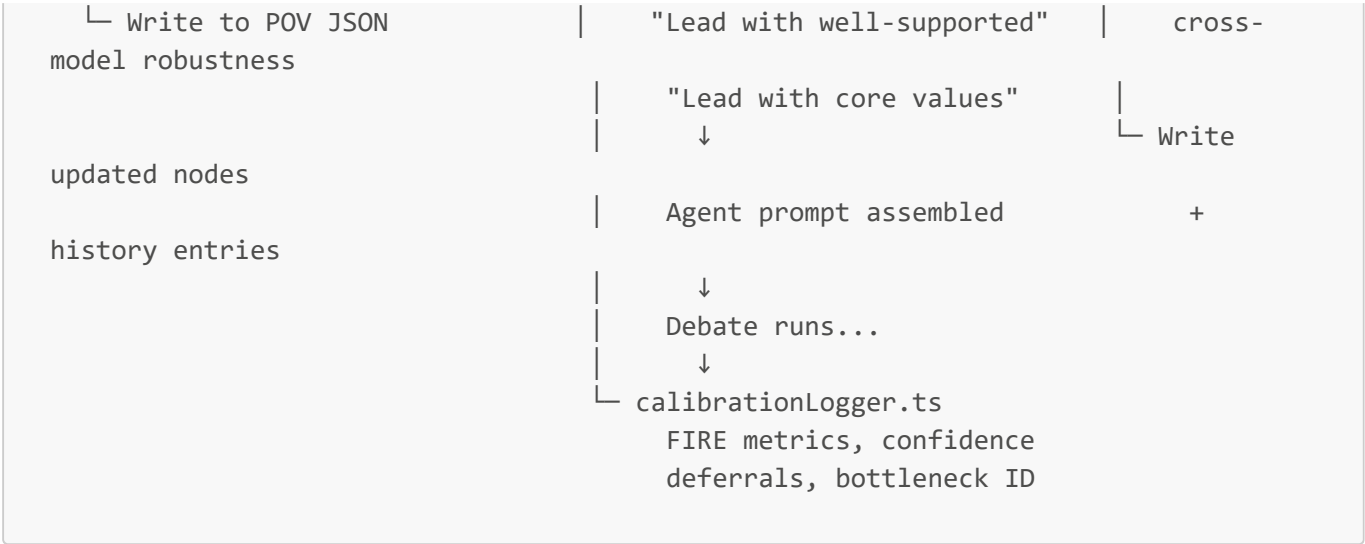
The debate engine uses two category-specific weights to control how taxonomy nodes influence agent behavior:

- **Belief Confidence** (0.0–1.0) — evidential strength of empirical claims
- **Desire Priority** (1–5) — normative importance of value commitments

These weights affect node sorting in prompts, inline labeling, agent framing instructions, and post-debate evolution. This document traces the complete data flow.

Pipeline Summary

INITIALIZATION	DEBATE EXECUTION	POST-DEBATE
assignWeights.ts confidenceEvolution.ts └─ beliefConfidence.ts condition gate base + evidence + (attribution >0.6 debate + edge boosts undermine → confidence [0.10,0.95] strength >0.5) └─ doctrinalAnchoring.ts computeUndermineDelta cosine > 0.55 → 0.10 floor at 0.60 ±0.30 └─ desirePriority.ts tree position → 2-4 confidenceDedup.ts doctrinal boundary → 5 dedup (cos>0.80) dedup (cos>0.85)	debateEngine.ts └─ taxonomyContext.ts weightedScore() nodeWeightLabel() formatTaxonomyContext() ↓ Sort: rel × confidence (B) rel × priority/5 (D) ↓ Label: "(confidence: 0.82)" ↓ "(priority: 4/5)" "[Speculative, 0.35]" ↓ Instruction:	└─ 3- + + └─ ±0.05- └─ drift cap topic attack



Stage 1: Initialization

1.1 Belief Confidence (`beliefConfidence.ts`)

`computeBeliefConfidence(signals)` applies a multi-signal formula:

```
confidence = base + evidenceBoost + debateBoost + edgeBoost
            → clamped [0.10, 0.95]
```

Signal	Source	Range
Base	<code>epistemic_type</code> × <code>falsifiability</code> from graph attributes	0.40–0.80
Evidence boost	+0.05 per unique source doc (capped +0.15)	0–0.15
Debate boost	+0.03 per prior debate reference (capped +0.10)	0–0.10
Edge boost	+0.02 per support edge, -0.02 per attack (capped ±0.05)	-0.05–+0.05

Base scores by epistemic type:

Epistemic Type	High Falsifiability	Medium	Low
<code>empirical_claim</code>	0.80	0.70	0.60
<code>predictive</code>	0.40	0.40	0.40
<code>interpretive_lens</code>	0.50	0.50	0.50
<code>definitional</code>	0.50	0.50	0.50

`assignBeliefConfidences()` runs this for all Belief nodes in a POV, sets `node.confidence` and creates the first `confidence_history` entry.

1.2 Doctrinal Anchoring (`doctrinalAnchoring.ts`)

Prevents core POV beliefs from being scored as speculative merely due to low source coverage.

1. Embed each POV's 4 doctrinal boundary strings (all-MiniLM-L6-v2, 384-dim)
2. For each Belief: compute cosine similarity to all boundary vectors
3. If max similarity $\geq 0.55 \rightarrow \text{doctrinally_anchored} = \text{true}$
4. If anchored AND confidence < 0.60 :
 - Save original as `evidential_confidence`
 - Raise `confidence` to the floor (0.60)

1.3 Desire Priority (`desirePriority.ts`)

`assignDesirePriorities()` scores based on tree position and doctrinal status:

Condition	Priority	Label
Doctrinal boundary	5	Core
Root node (no parent)	4	High
Mid-tree (parent + children)	3	Medium
Leaf node (parent, no children)	2	Low

1.4 Orchestration (`assignWeights.ts`)

CLI entry point: `npm run tsx lib/debate/assignWeights.ts [--dry-run]`

Runs all three computations per POV, writes updated nodes back to disk. Outputs summary stats: avg/min/max confidence, anchoring counts, priority distribution.

Stage 2: Storage

PovNode Fields (`taxonomyTypes.ts`)

```
// Beliefs only
confidence?: number; // [0.0, 1.0]
confidence_history?: WeightHistoryEntry[];
doctrinally_anchored?: boolean;
evidential_confidence?: number; // Pre-floor value when anchored

// Desires only
priority?: number; // 1-5
priority_history?: WeightHistoryEntry[];
```

Weight History Entry

```
interface WeightHistoryEntry {
  date: string; // ISO 8601
  value: number; // New value
}
```

```

    delta: number;           // Change from prior
    reason: string;          // "Initial multi-signal assignment" or "Debate X:
undermined"
    attack_claim?: string;    // AN claim text that triggered change
    supersedes?: string;      // Debate ID of prior update (dedup)
    robustness?: number;      // Models confirming (≥2 = cross-model)
    model_confirmations?: string[];
  }

```

Stage 3: Prompt Injection

Weighted Sorting ([taxonomyContext.ts:88-97](#))

```

function weightedScore(node, relevance, category) {
  if (Beliefs)    → relevance × confidence
  if (Desires)    → relevance × (priority / 5)
  if (Intentions) → relevance alone
}

```

High-confidence Beliefs and high-priority Desires sort to the top of their BDI sections.

Inline Labels ([taxonomyContext.ts:100-111](#))

Condition	Label
Belief, confidence ≥ 0.50	(confidence: 0.82)
Belief, confidence < 0.50	[Speculative, confidence: 0.35]
Belief, doctrinally anchored	(confidence: 0.60, doctrinally anchored)
Desire	(priority: 4/5)
Intention	(no label)

Framing Instructions ([taxonomyContext.ts:148-152](#))

When weighted data is present, the formatter injects category-specific instructions:

- **Beliefs:** "Ordered by evidential confidence. Lead with well-supported claims. When you cite a low-confidence Belief, acknowledge the uncertainty explicitly."
- **Desires:** "Ordered by priority. Lead with non-negotiable values. Lower-priority desires may be traded off in argument."

Example Prompt Output

```

=== YOUR EMPIRICAL GROUNDING (what you take as true) ===
Ordered by evidential confidence. Lead with well-supported claims.

```

```
★ [acc-bel-042] (relevance: 0.71) (confidence: 0.82) Inherent Power-Seeking: ...
★ [acc-bel-015] (relevance: 0.68) (confidence: 0.75, doctrinally anchored) Market
Efficiency: ...
  [acc-bel-088] (relevance: 0.52) [Speculative, confidence: 0.35] Rapid Capability
Emergence: ...

=== YOUR NORMATIVE COMMITMENTS (what you argue should happen) ===
Ordered by priority. Lead with non-negotiable values.

★ [acc-des-001] (relevance: 0.80) (priority: 5/5) Innovation Freedom: ...
★ [acc-des-013] (relevance: 0.65) (priority: 4/5) Long-Term AI Benefit: ...
  [acc-des-027] (relevance: 0.48) (priority: 2/5) Cost Reduction: ...
```

Calibration Audit (prompts.ts:2121-2172)

When belief confidences are available, the extraction/evaluation prompt includes a calibration block that asks the LLM to verify whether the draft's rhetoric matches evidential strength — citing a low-confidence Belief as "established fact" is flagged as a weakness.

Stage 4: Mid-Debate Evolution

Confidence Updates (confidenceEvolution.ts)

After a debate completes, the engine evaluates whether Belief confidences should change.

Three-condition gate (all must pass to reduce confidence):

- 1. attribution_confidence > 0.60 — the AN claim clearly instantiates the Belief
- 2. attack_type === 'undermine' — the attack targets evidential foundations (not just conclusions)
- 3. attack_strength > 0.5 — the attack is strong enough to matter

Update magnitude (computeUndermineDelta):

Claim Outcome	Strength	Delta
Claim survived attacks	> 0.7	+0.05
Claim in contested middle	0.3–0.7	0
Claim fell below viability	< 0.3	-0.05 to -0.10

Drift cap: Total change from initial assignment capped at ±0.30 (MAX_DRIFT). Prevents a single debate from radically altering long-held positions.

Additional signals:

- Cross-POV validation (opposing camp cites Belief as evidence): **+0.10**
- Document contradiction discovered: **-0.15**

Priority Updates

- **Concession during debate:** reduce priority by 1
- **Identified as crux of disagreement:** increase priority by 1

Cross-Debate Deduplication ([confidenceDedup.ts](#))

Prevents multi-model debate runs from double-penalizing the same Belief.

Check	Similarity Threshold	Behavior
Same topic	cosine > 0.80	Take max magnitude; discard weaker
Same attack vector, same model	cosine > 0.85	Replace if stronger; discard if weaker
Same attack vector, different model	cosine > 0.85	Robustness signal — record both models confirming

Cross-model robustness (≥ 2 models confirming the same attack) is recorded in the history entry as `robustness: N` and `model_confirmations: [...]`.

History Pruning

Keep last 30 entries OR entries ≤ 12 months old. Track pruned net delta for audit trail.

Stage 5: Diagnostics Surfacing

DiagnosticsWindow — Confidence Impact Panel ([DiagnosticsWindow.tsx:3119-3175](#))

After a debate loads, scans all POV taxonomy nodes for `confidence_history` and `priority_history` entries matching the debate ID. Renders a panel showing:

Confidence Impact (5)				
acc-bel-042	Inherent Power-Seeking	0.82	+0.05	Survived attacks
saf-des-015	Mitigating Displacement	3	-1	Concession
skp-bel-068	Long-Horizon Uncertainty	0.45	-0.08	Undermined

- Delta colored green (+), red (-), or gray (0)
- Robustness badge: "2× confirmed" for cross-model agreement
- Attack claim snippet shown for undermine-driven changes

Node Detail View ([GraphAttributesPanel.tsx](#))

Belief nodes:

- Confidence slider (0.00–1.00), color-coded: green ≥ 0.70 , blue 0.40–0.69, orange < 0.40
- Doctrinal anchor badge with tooltip explaining the floor mechanism
- Evidential confidence display when floor was applied: "Evidential: 0.35 (floor applied: 0.60)"
- History summary: "3 update(s) — latest: Debate d-2024-11-18: survived"

Desire nodes:

- Priority dropdown (1–5)
- P5 flagged: "Core — doctrinal boundary"
- History summary with latest reason

Calibration Log (`calibrationLogger.ts`)

Each debate session appends to `calibration-log.json`:

- `fire_confidence_threshold` — current FIRE acceptance threshold
- `confidence_deferrals` — times extraction confidence was deferred
- `confidence_bottleneck` — which signal bottlenecked ('extraction' | 'stability' | 'none')
- `fire_survived_rate` — fraction of borderline (0.5–0.7) claims that survived debate
- Used by the calibration optimizer for real-time threshold auto-tuning

File Reference

File	Key Functions	Stage
<code>beliefConfidence.ts</code>	<code>computeBeliefConfidence()</code> , <code>assignBeliefConfidences()</code>	Init
<code>desirePriority.ts</code>	<code>computeTreePriority()</code> , <code>assignDesirePriorities()</code>	Init
<code>doctrinalAnchoring.ts</code>	<code>computeDoctrinalAnchoring()</code>	Init
<code>assignWeights.ts</code>	CLI orchestrator	Init
<code>taxonomyTypes.ts:59-86</code>	<code>PovNode</code> fields, <code>WeightHistoryEntry</code>	Storage
<code>taxonomyContext.ts:88-190</code>	<code>weightedScore()</code> , <code>nodeWeightLabel()</code> , <code>formatTaxonomyContext()</code>	Prompt
<code>prompts.ts:2121-2172</code>	Confidence calibration audit block	Prompt
<code>confidenceEvolution.ts</code>	<code>evaluateGate()</code> , <code>computeUndermineDelta()</code> , <code>pruneHistory()</code>	Evolution
<code>confidenceDedup.ts</code>	<code>evaluateDedup()</code> , cross-model robustness	Evolution
<code>DiagnosticsWindow.tsx:3119-3175</code>	Confidence impact panel	Diagnostics
<code>GraphAttributesPanel.tsx</code>	Sliders, badges, history summary	Diagnostics
<code>calibrationLogger.ts</code>	FIRE metrics, confidence deferrals	Diagnostics

Design Principles

1. **Weights are signals, not gates.** Low confidence doesn't block a Belief from injection — it changes its sort position and label. The agent sees it and can choose to cite it with appropriate hedging.

2. **Doctrinal anchoring prevents epistemic bootstrapping.** A core POV belief shouldn't be scored as "speculative" just because it has fewer source docs than an empirical claim. The floor ensures doctrinal positions maintain minimum presence.
3. **Evolution is conservative.** The 3-condition gate + drift cap ensures that confidence changes are evidence-driven and bounded. A single bad debate can't destroy a well-supported Belief.
4. **Cross-model robustness adds signal, not penalty.** When two different models independently confirm the same attack on a Belief, that's recorded as a robustness signal — stronger evidence than a single model's judgment.
5. **Priority is structural, not evidential.** Desire priority comes from tree position and doctrinal status, not from how many sources mention it. This reflects the normative nature of Desires — they're commitments, not hypotheses.